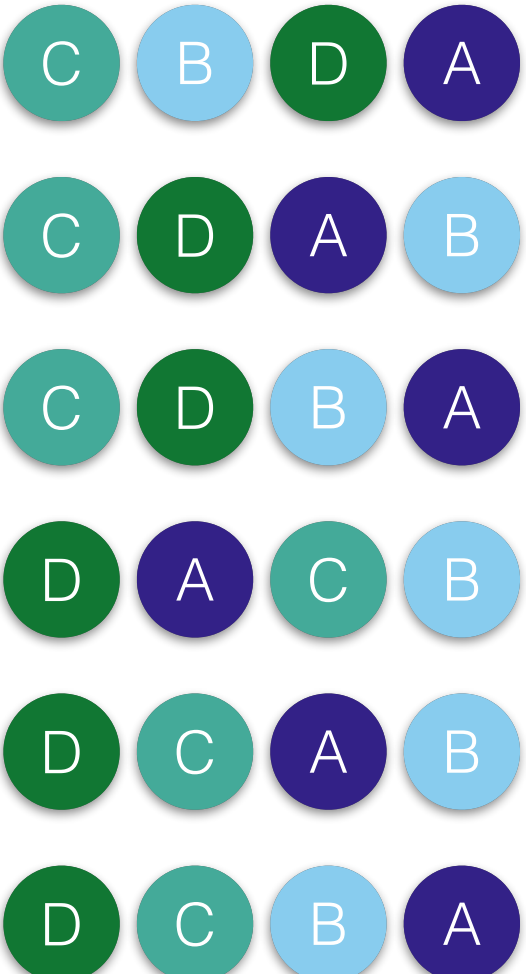
What does this get us?

```
when(src) { A src.balance += 10; }
when(dst) { B dst.balance += 10; }
```

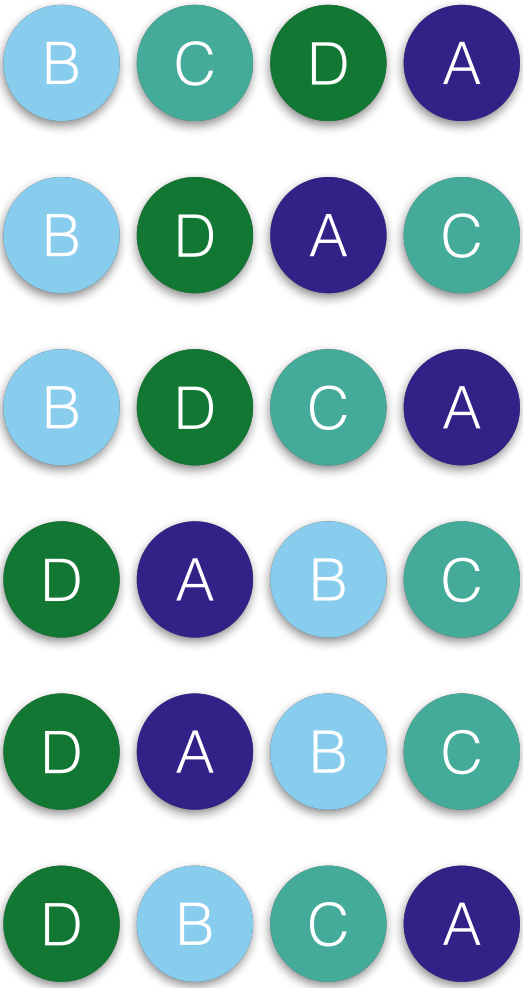
46

```
when(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }
```

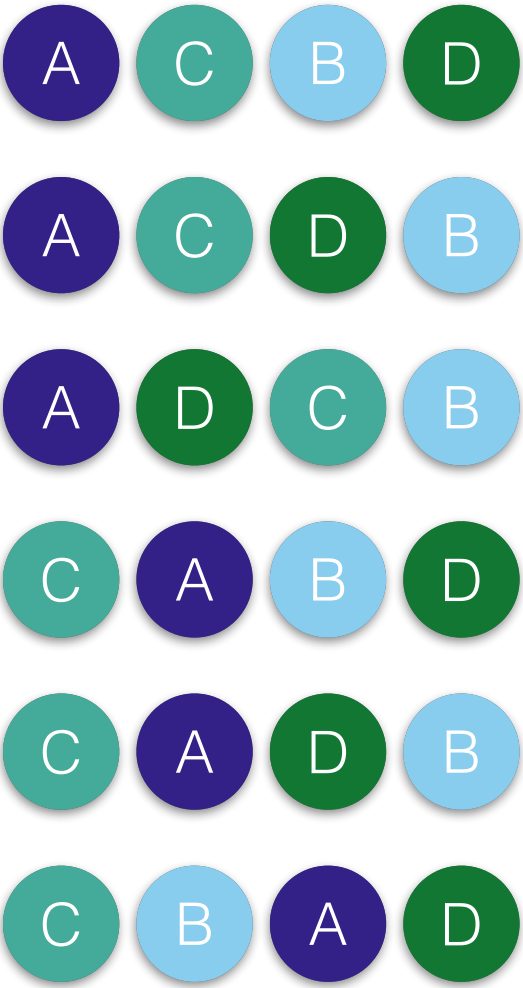
src=0, dst=0



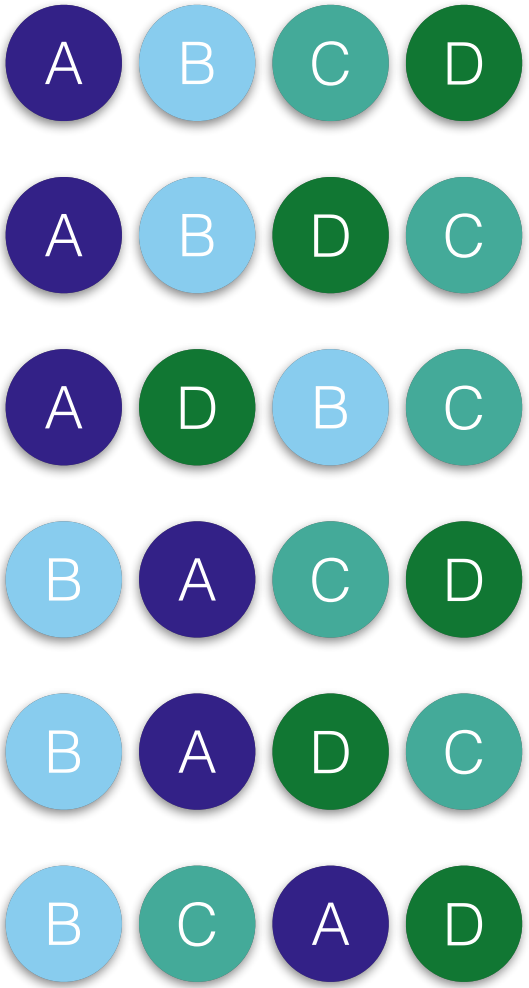
src=0, dst=10



src=10, dst=0



src=10, dst=10

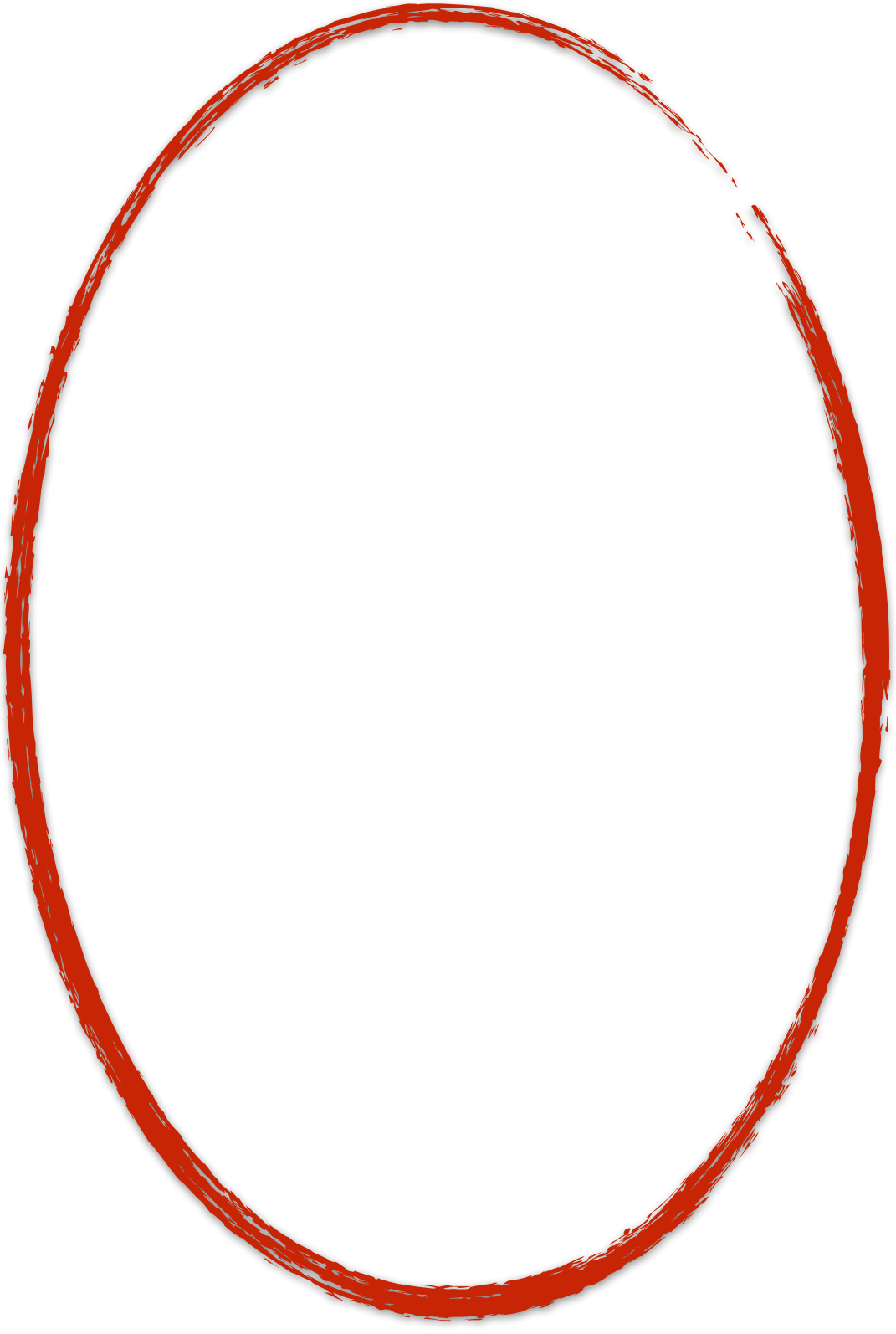


Order

Permitted

Update path (`src` then `dst`)

Diagnostic path (`dst` then `src`)



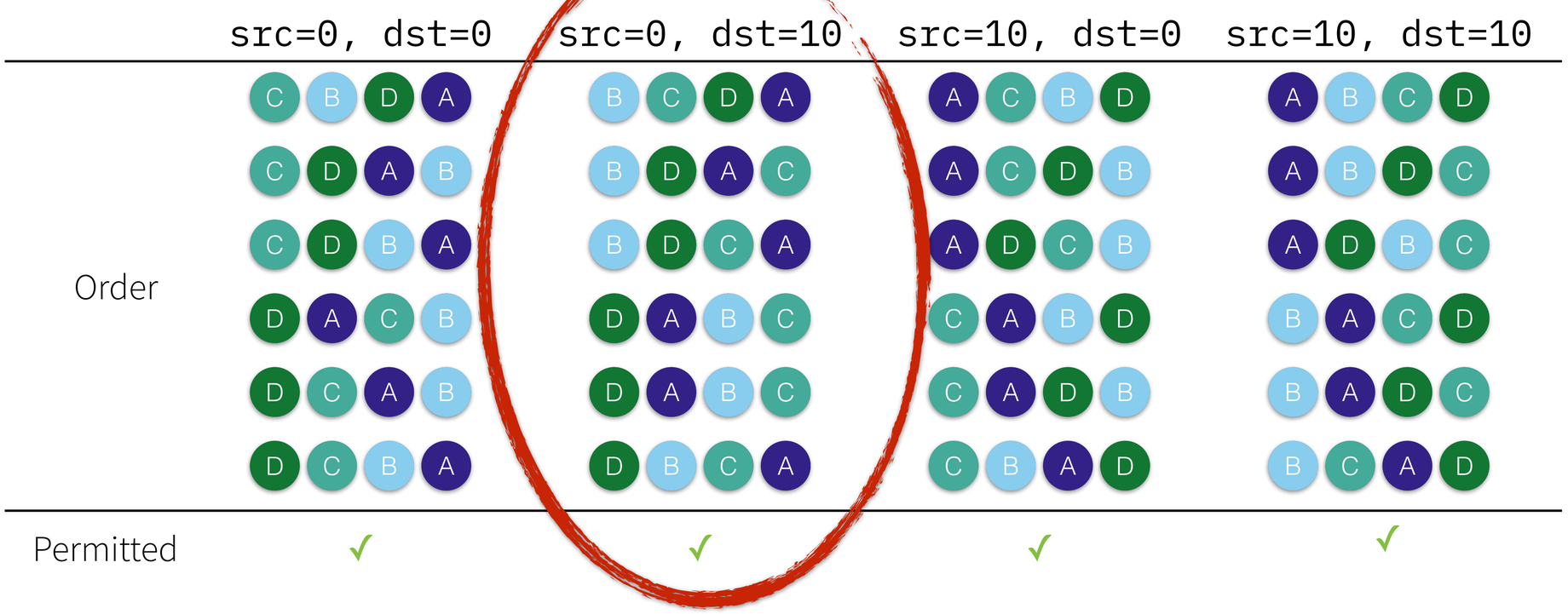
What does this get us?

Update path (src then dst)

```
when(src) { A src.balance += 10; }
when(dst) { B dst.balance += 10; }
```

Diagnostic path (dst then src)

```
when(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }
```

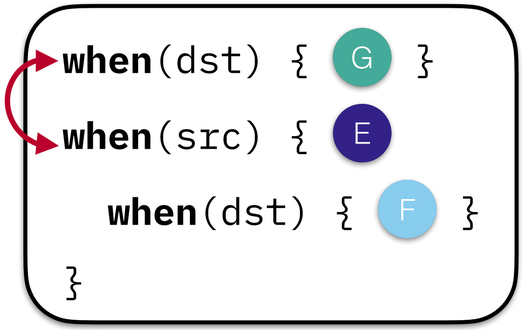


What does this get us?

We can reason about program transformations

```
when(busy, idle) { A
  val r = busy.use();
  idle.use(r)
  when(log) { B }
  idle.use(r)
}
when(busy) { C
  when(log) { D }
}
```

Can we release cowns early?



Can we reorder behaviours?